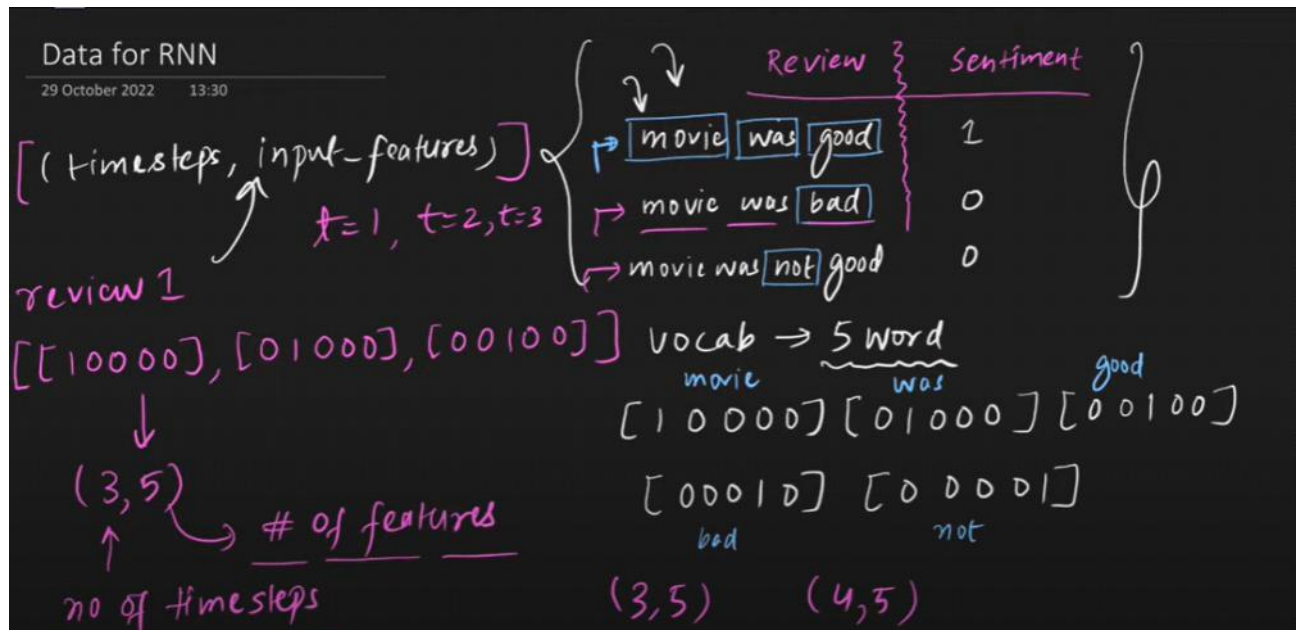


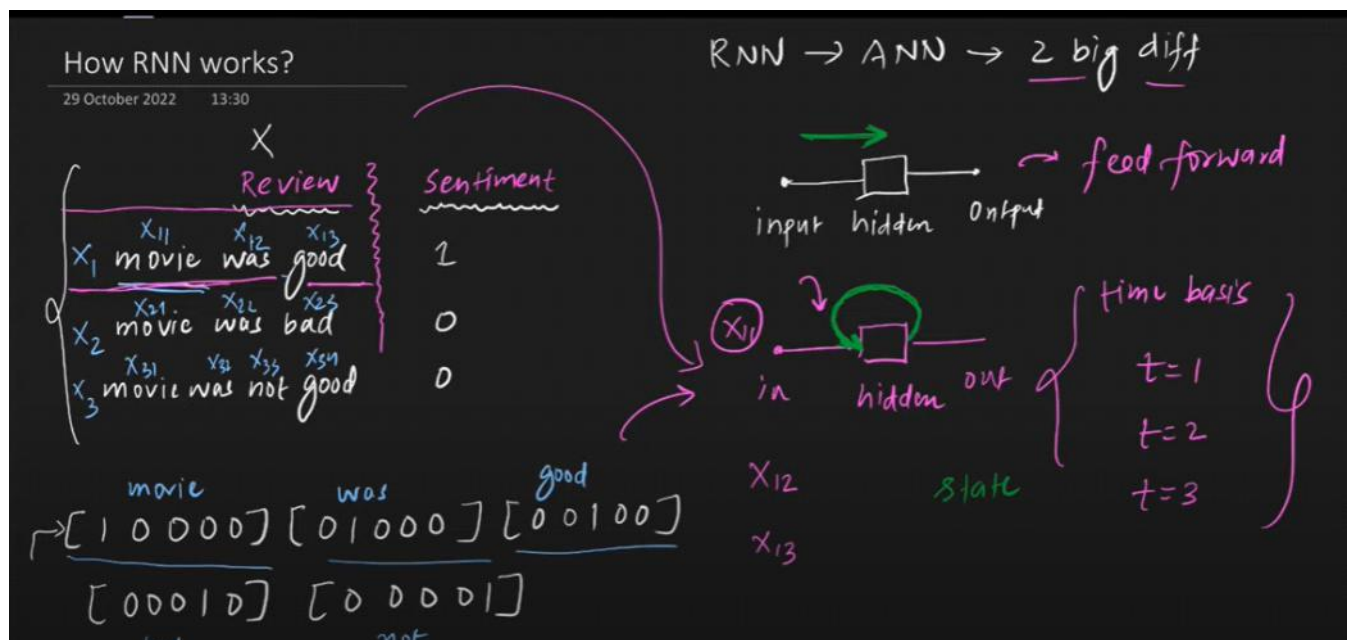
RNN

08 July 2024 20:35

Shape of each line of reviews (timesteps, total input vocabulary) e.g. below 5 words are there and timestep is 3 as the 1st sentence has 3 words
So T1, T2, T3 each words will be passed

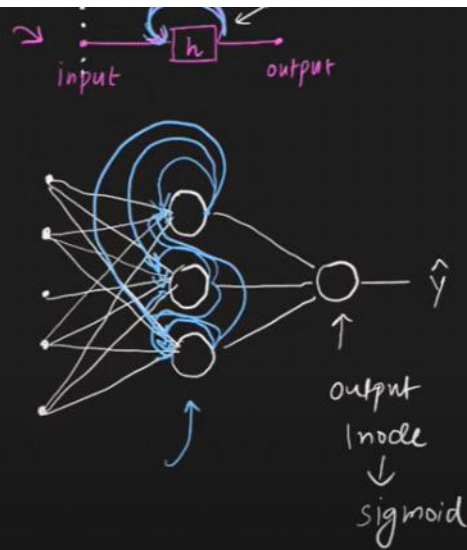
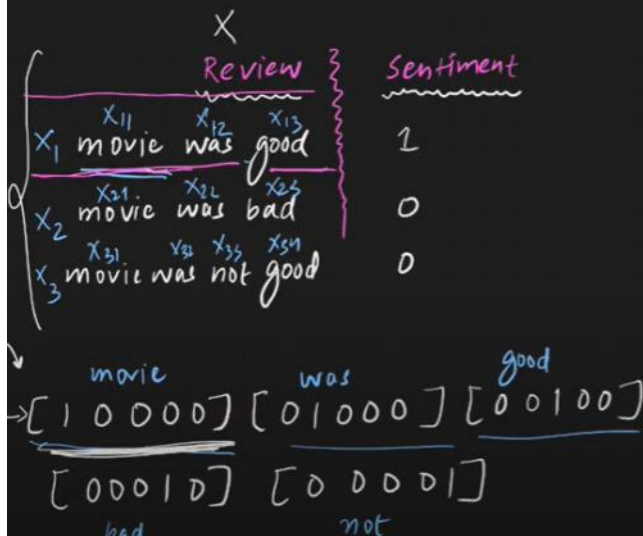


Keras $\rightarrow$ Simple RNN $\rightarrow$ batch size, timesteps, input-features



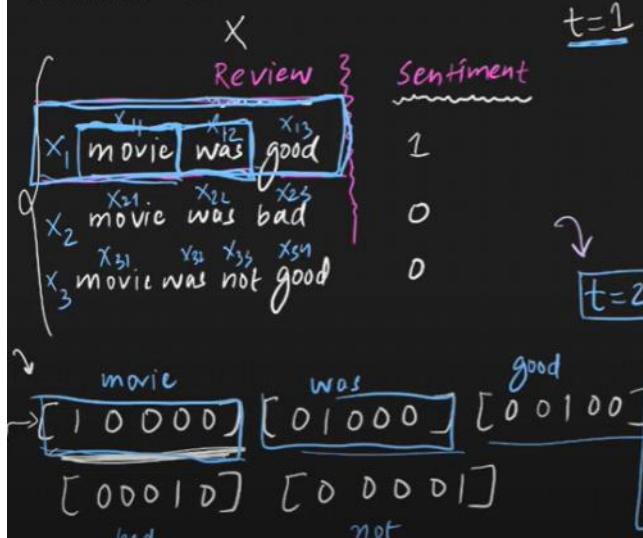
How RNN works?

29 October 2022 13:30



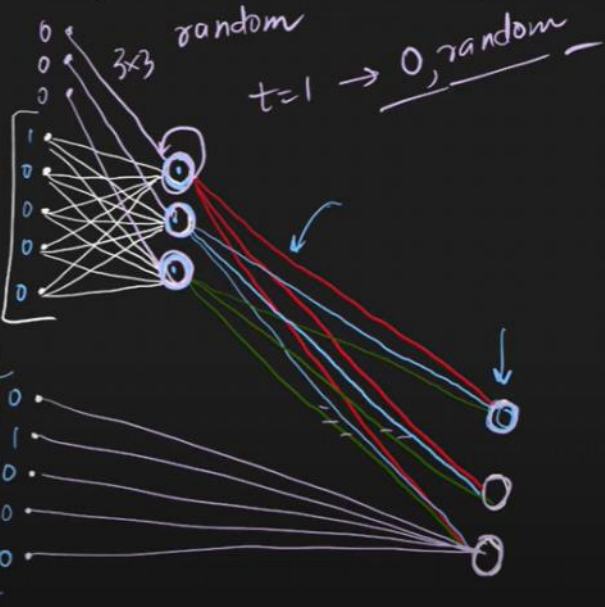
How RNN works?

29 October 2022 13:30



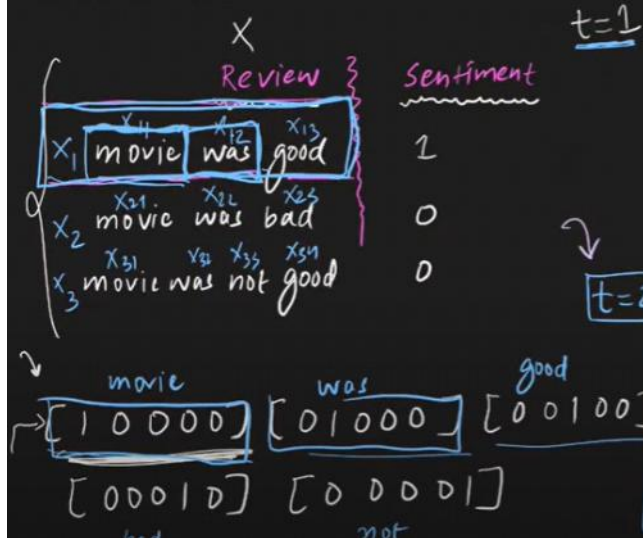
$t=1$

$t=2$



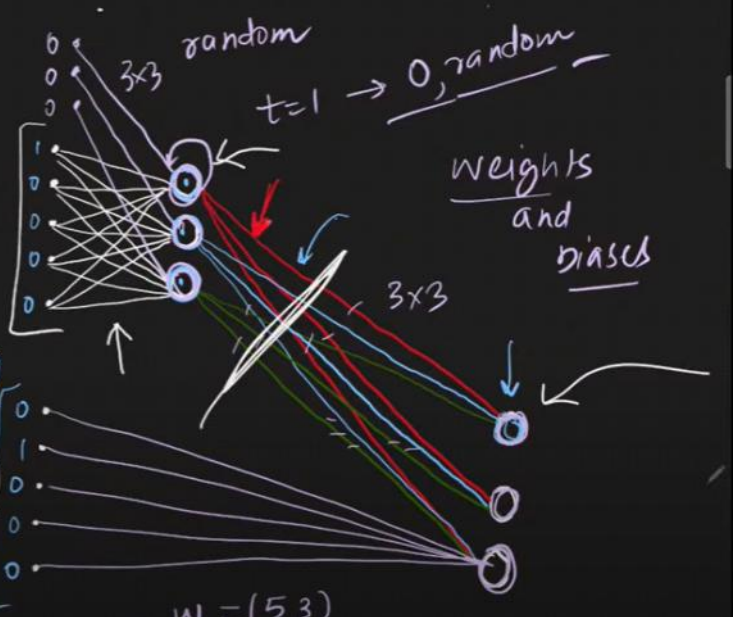
RNN Architecture

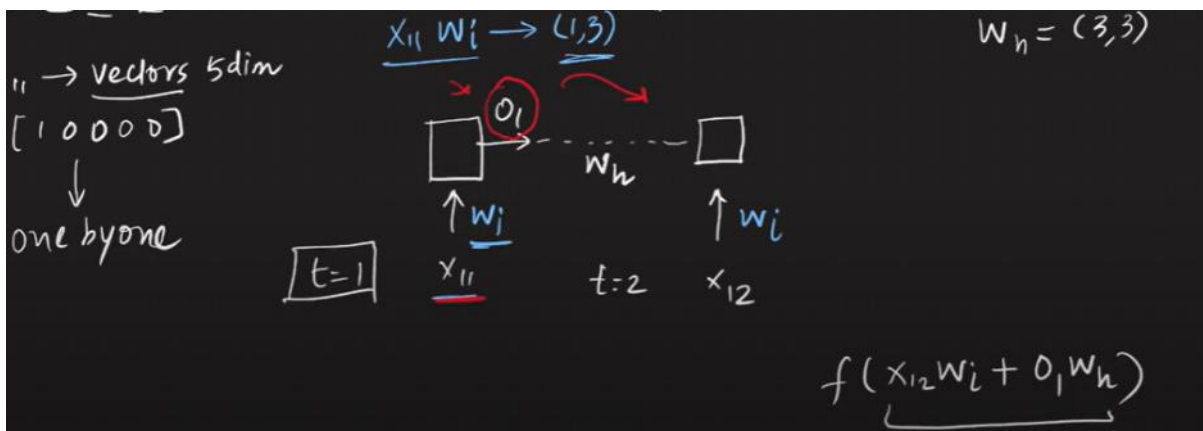
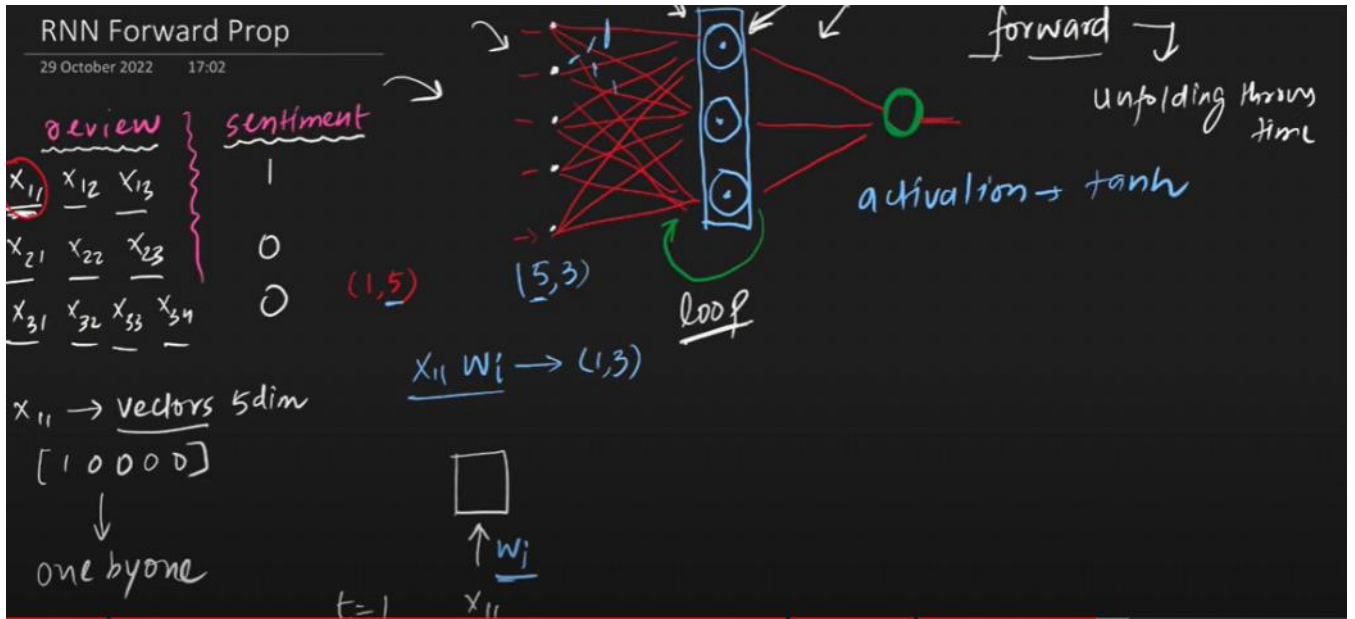
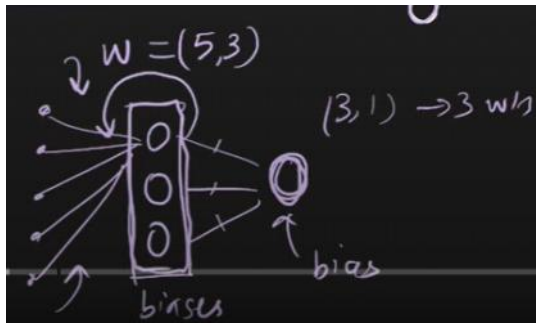
29 October 2022 13:30



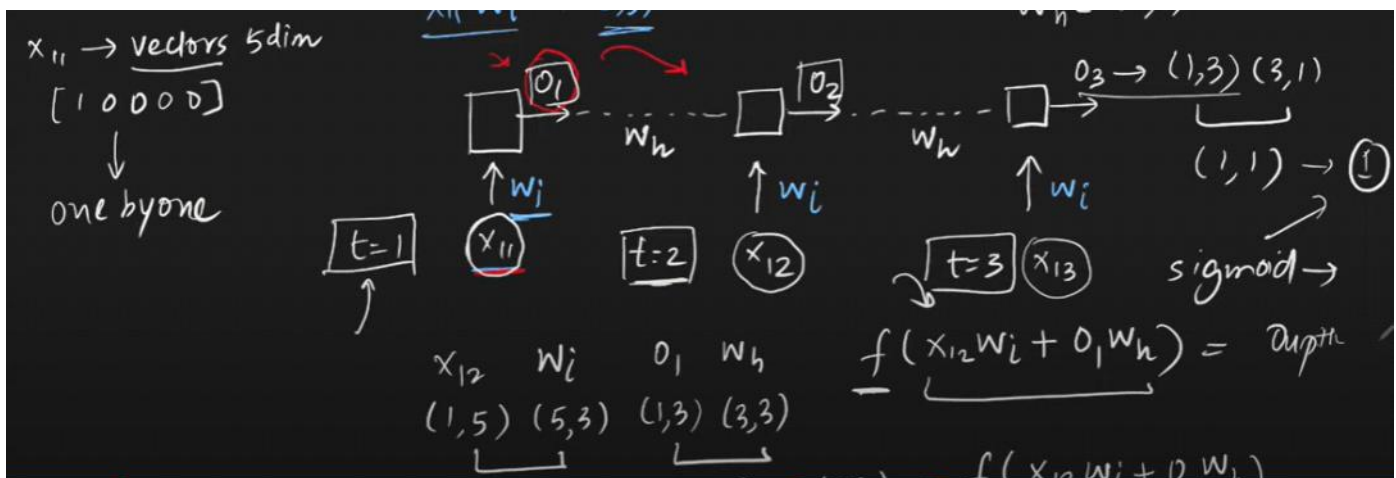
$t=1$

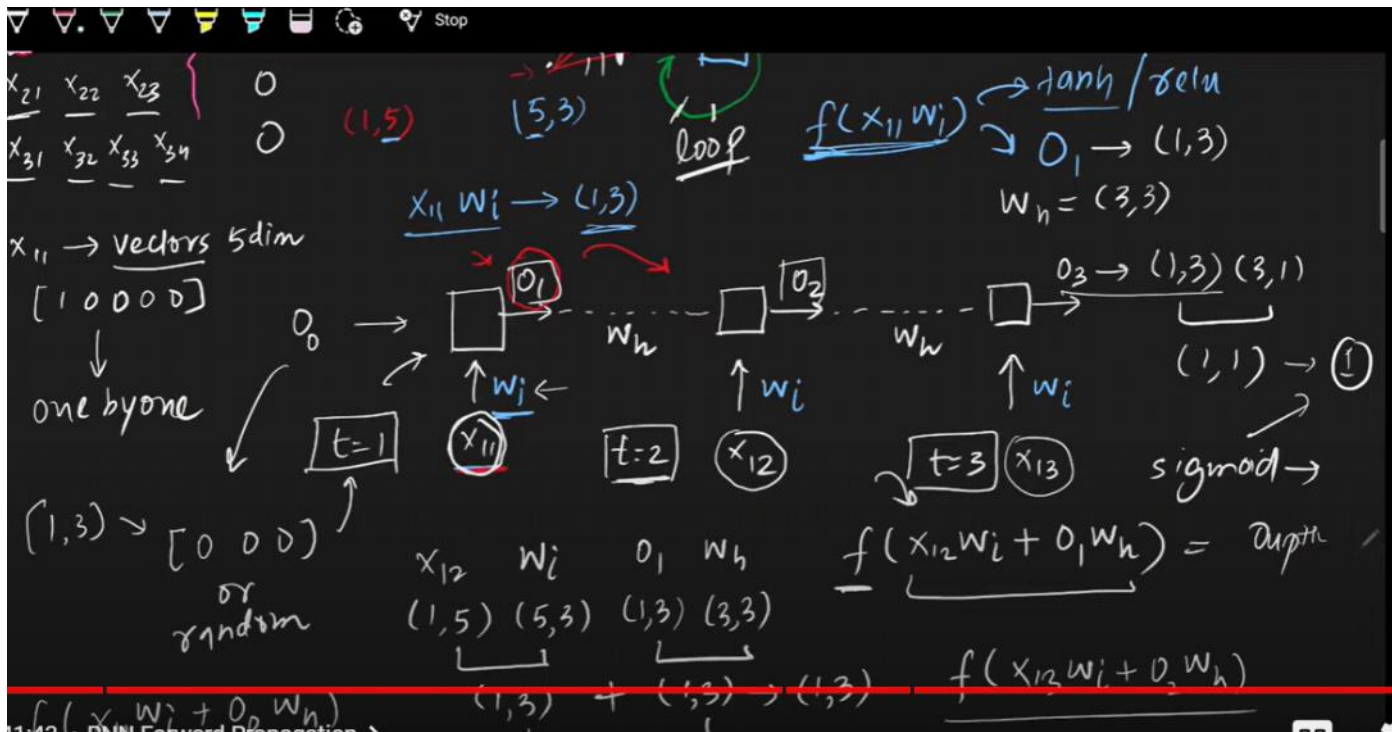
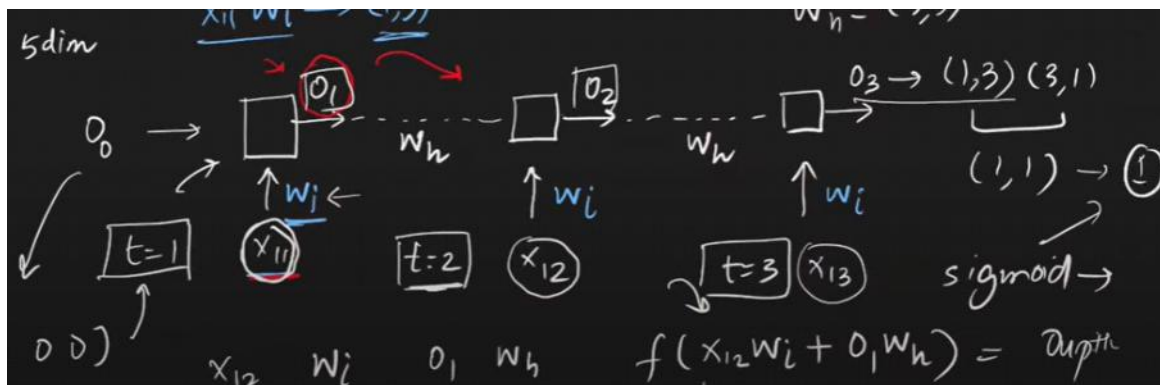
$t=2$



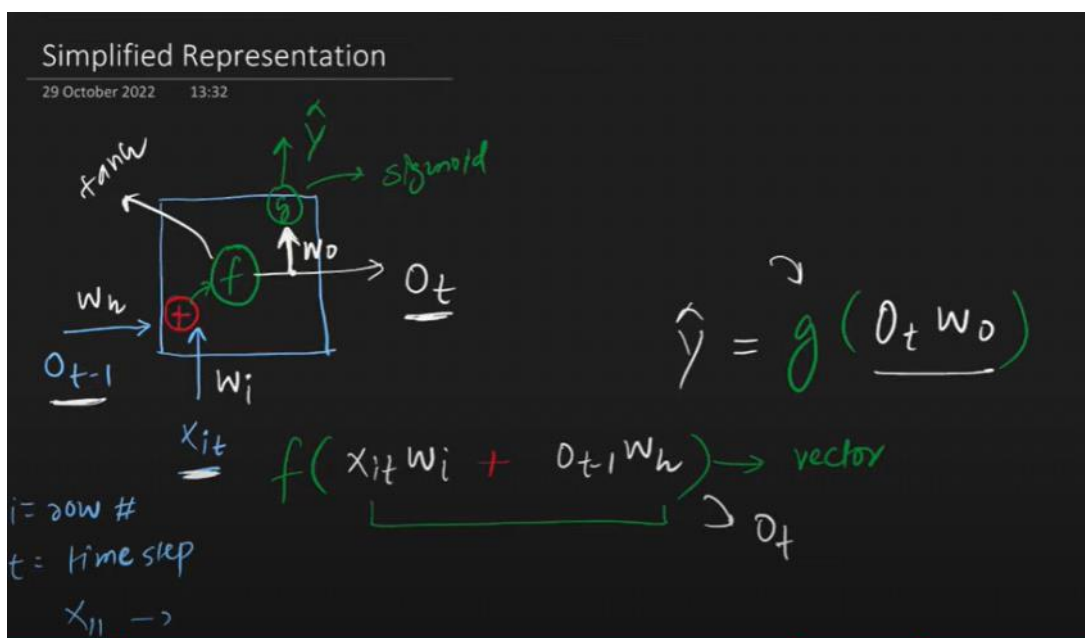


W_h - is the weighted connection associated with the output (recurrent) loop which is also the input in the $T+1$ timestep





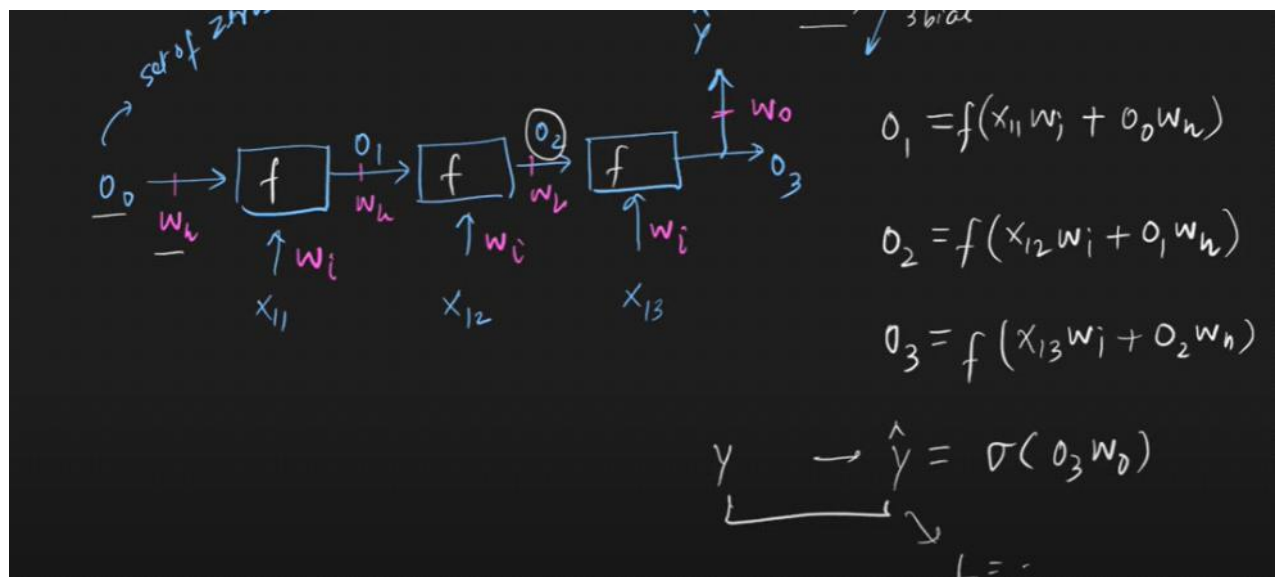
00 (Ozero)- is the initial dummy (0,0,0,0) output which are the input just to maintain consistency .



G = activation function for the final layer. Hence it can be sigmoid as the activation function

Inside RNN which is a Neural network and each node has **tanh** as the activation function.
Note: A SimpleRNN can process and store information of previous 10 timesteps as memory.

Backpropagation in RNN



Loss function: $L = -y_i \log y_i - (1 - y_i) \log(1 - y_i)$

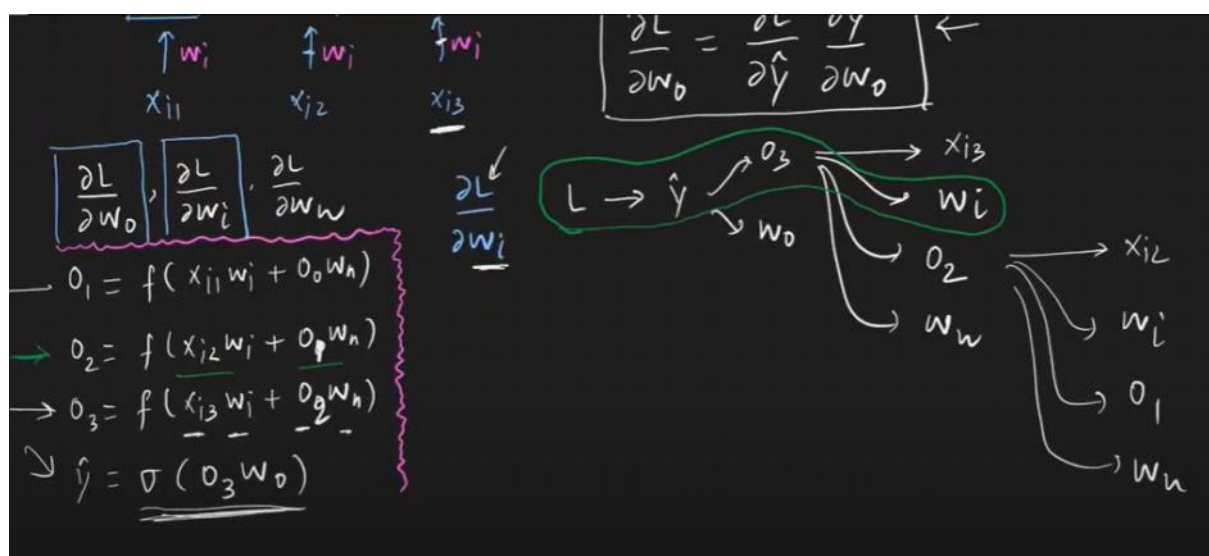
Minimize L using gradient descent:

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i}$$

$$w_h = w_h - \eta \frac{\partial L}{\partial w_h}$$

$$w_0 = w_0 - \eta \frac{\partial L}{\partial w_0}$$

Loss function = Log Loss (in case of Sentiment Analysis use case)



$$o_2 = f(x_{i2}w_i + o_1w_n)$$

$$o_3 = f(x_{i3}w_i + o_2w_n)$$

$$\hat{y} = \sigma(o_3w_o)$$

10 words → 10 times

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial w_i} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial w_i} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial o_1} \frac{\partial o_1}{\partial w_i}$$

Since 3 times unfolded as data set has 3 words and hence the derivative of Loss with respect to $\hat{y}$ will have 3 different derivatives and they will be added.

$$\frac{\partial L}{\partial w_n} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial w_n}$$

$$\frac{\partial L}{\partial w_n} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial w_n} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial o_1} \frac{\partial o_1}{\partial w_n}$$

$$\boxed{\frac{\partial L}{\partial w_n} = \sum_{j=1}^n \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_j} \frac{\partial o_j}{\partial w_n}}$$

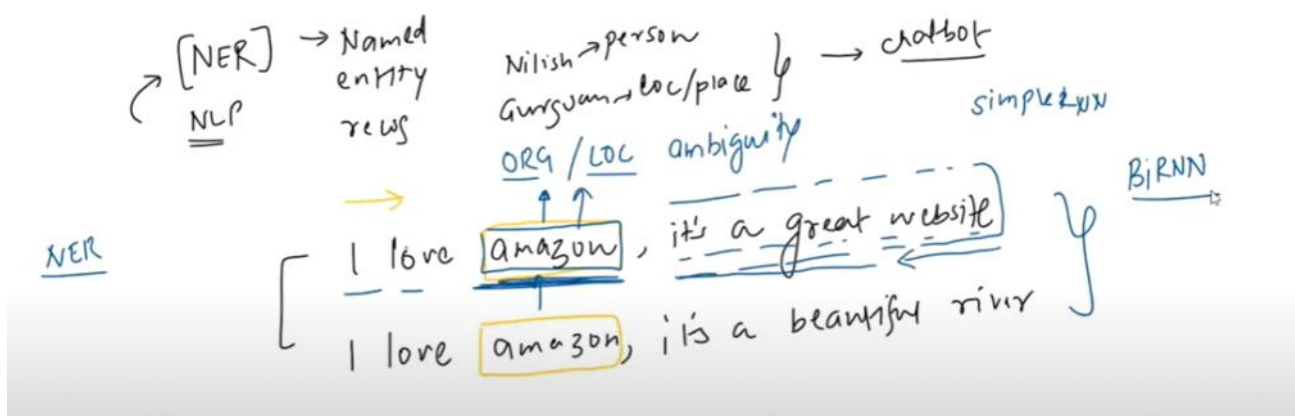
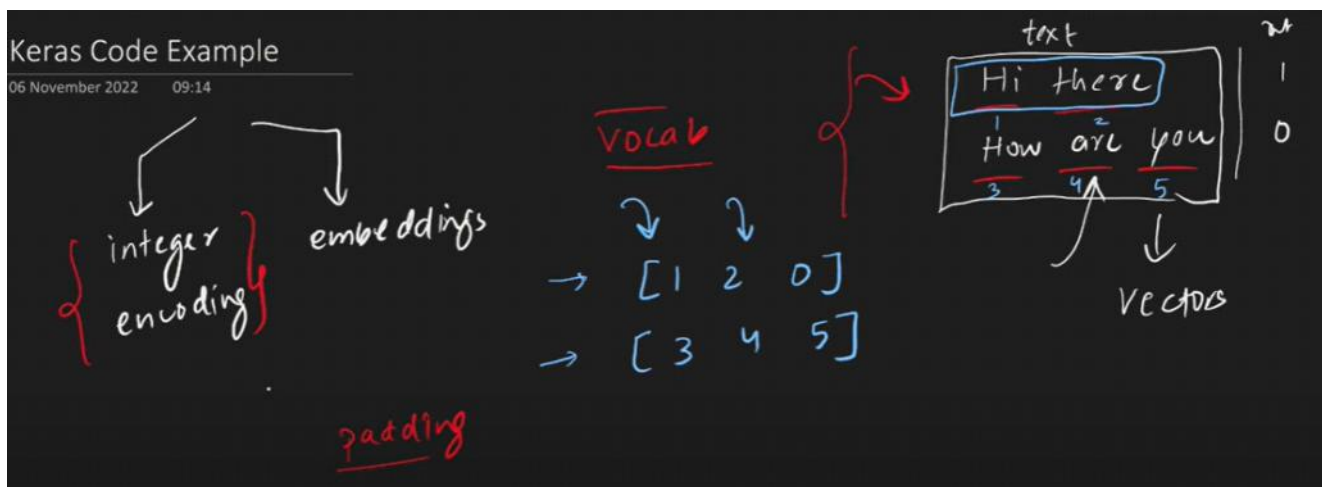
$n = \text{timesteps}$

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^3 \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_j} \frac{\partial o_j}{\partial w_i}$$

0 words
→ 10 runs
t=1

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^3 \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_j} \frac{\partial o_j}{\partial w_i}$$

$$= \frac{\partial L}{\partial \hat{y}} \left[\frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial w_i} \right] = \left[\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial o_1} \frac{\partial o_1}{\partial w_i} \right]$$



Bidirectional RNN Architecture

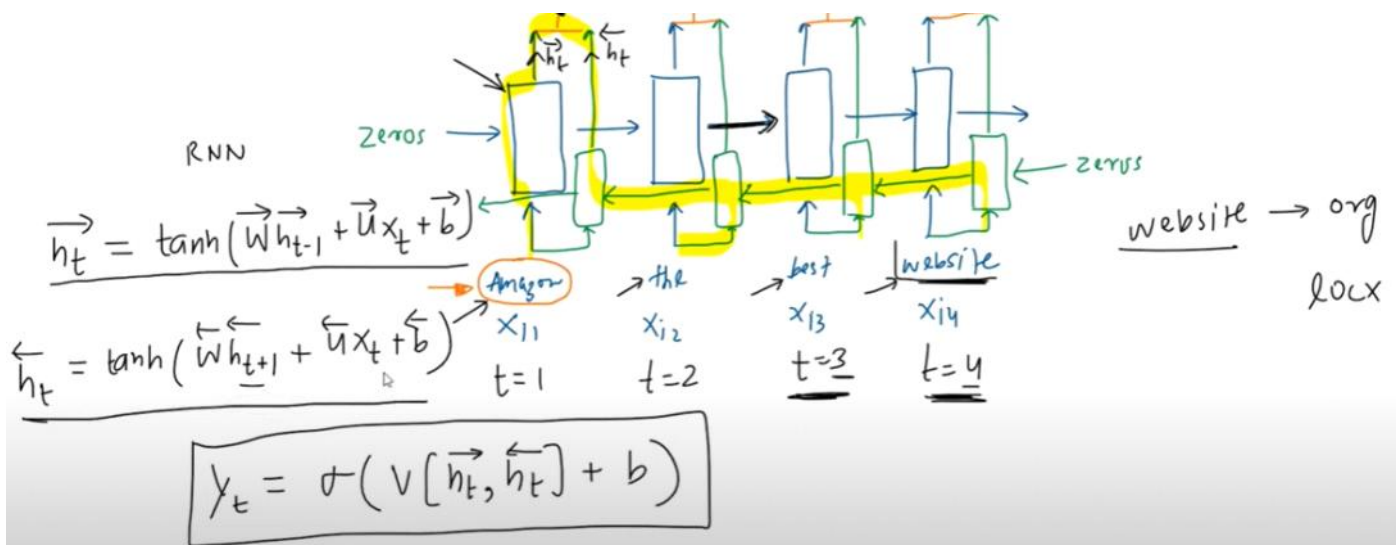
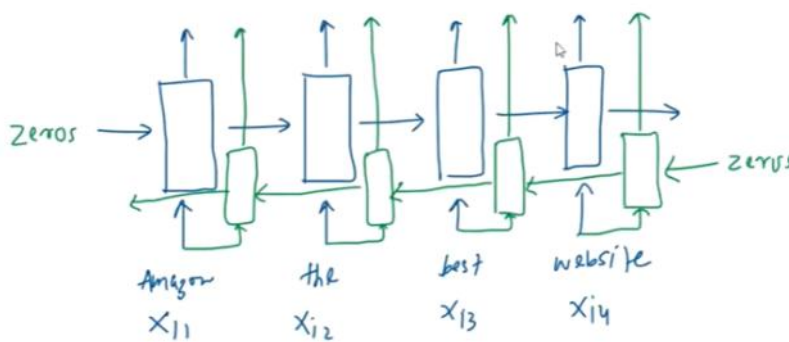
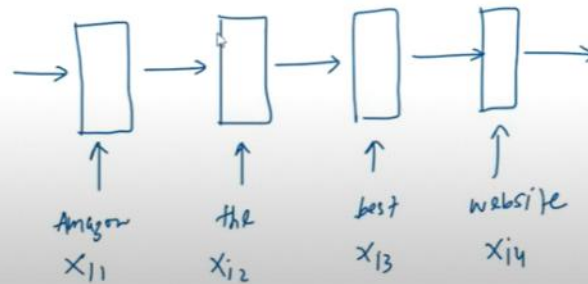
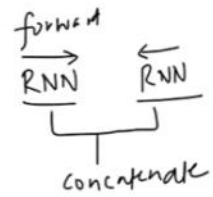
26 October 2023 15:19

Amazon the best website.
 Amazon the beautiful river.

→
 ←

loc / org

2 RNNs

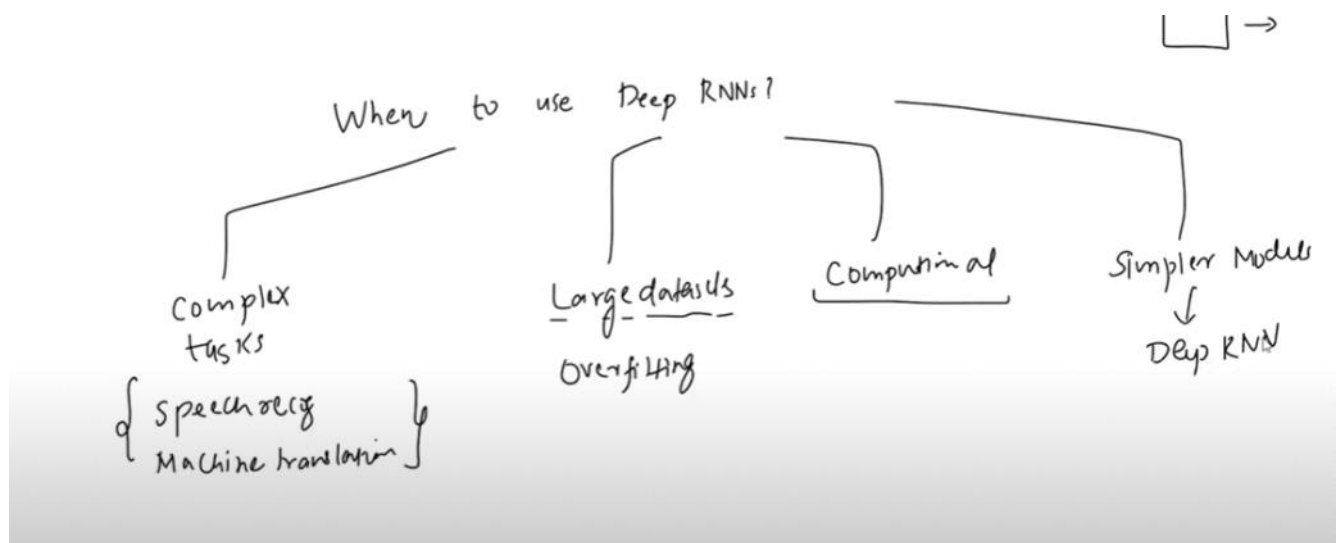
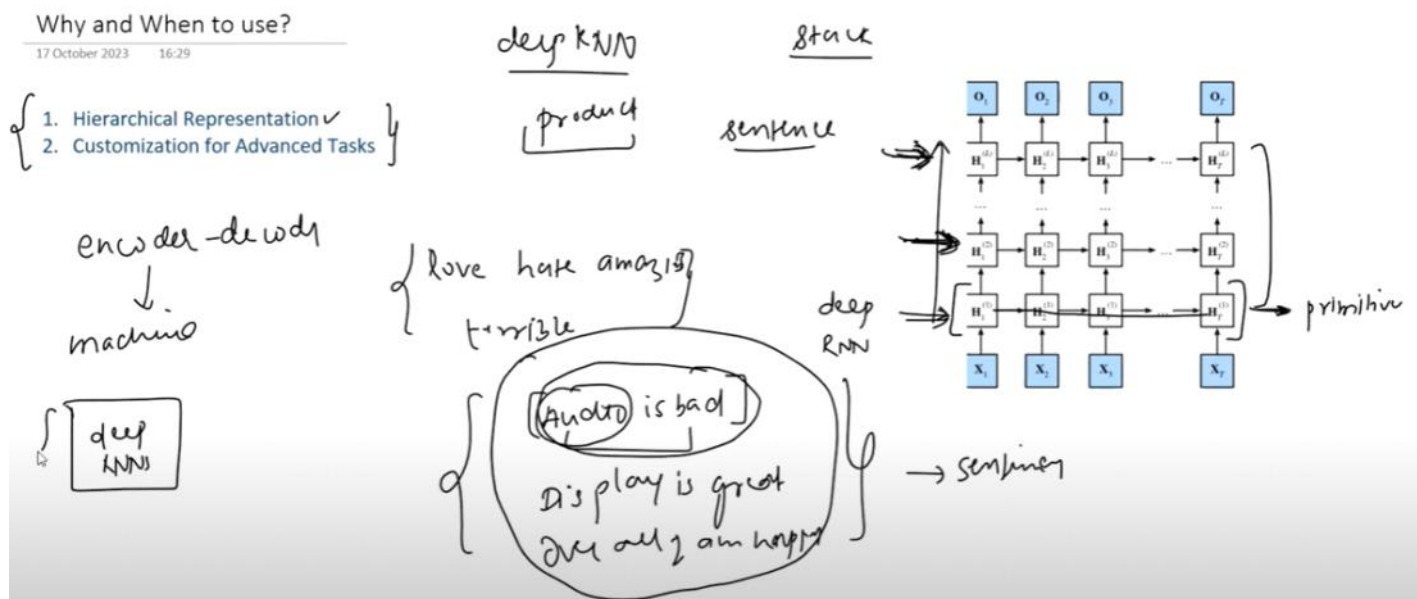
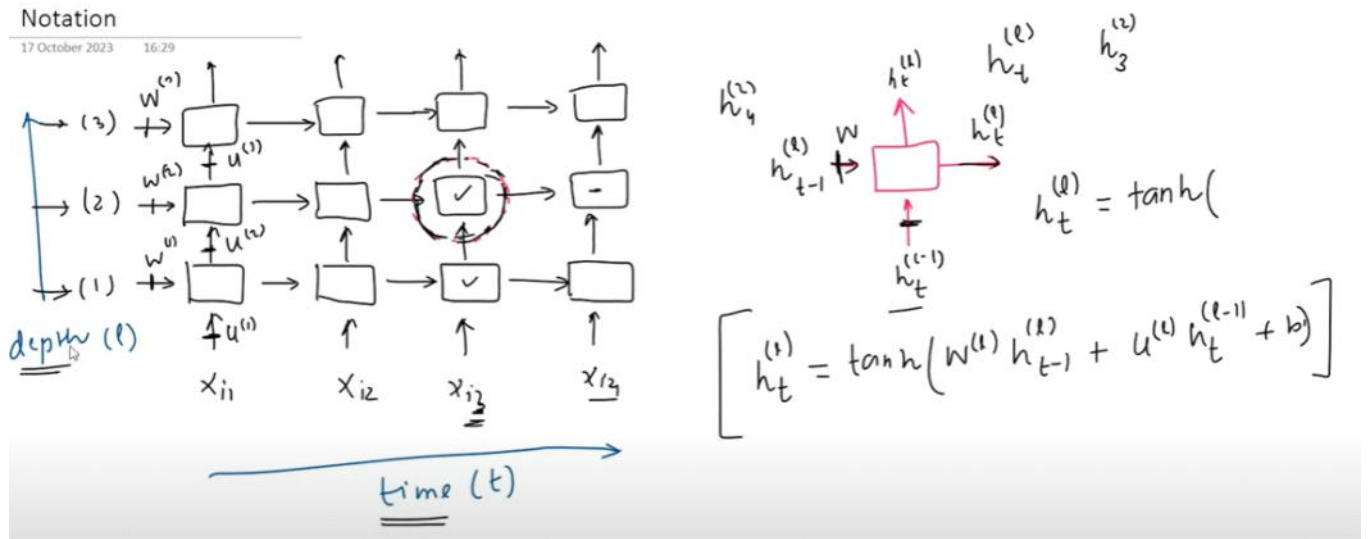


```
model = Sequential([
    Embedding(input_dim=num_words, output_dim=embedding_dim, input_length=maxlen),
    Bidirectional(SimpleRNN(5)), # 5 RNN units
    Dense(1, activation='sigmoid') # Binary classification (positive/negative)
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Display model architecture
model.summary()
```


Deep RNN as below:



```

[9] # Load the IMDB dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)

# Pad sequences to have the same length
x_train = pad_sequences(x_train, maxlen=100)
x_test = pad_sequences(x_test, maxlen=100)

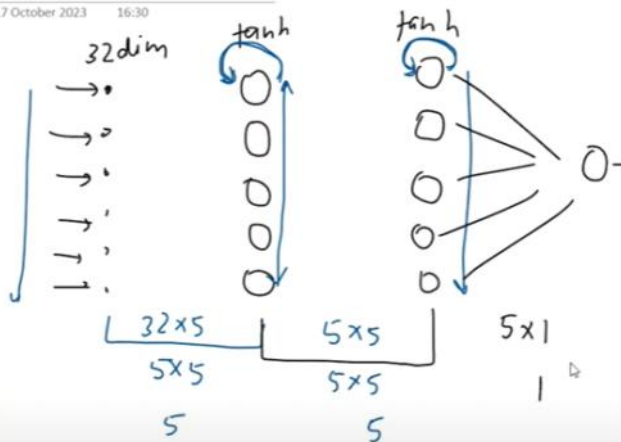
# Define the RNN model
model = Sequential([
    Embedding(10000, 32, input_length=100), # Embedding layer to convert words to vectors
    SimpleRNN(5, return_sequences=True),    # RNN layer with 5 units
    SimpleRNN(5),                          # Another RNN layer with 5 units
    Dense(1, activation='sigmoid')         # Output layer for binary classification
])

model.summary()

```

Code Example

17 October 2023 16:30



→ embedding

$$10000 \times 32 = 3,20,000$$

Simple KNN 1

190

Simple KNN 2

35

```

# Define the GRU model
model = Sequential([
    Embedding(10000, 32, input_length=100),
    GRU(5, return_sequences=True),
    GRU(5),
    Dense(1, activation='sigmoid')
])

model.summary()

```